

DBMS – SELLER AND INVENTORY MANAGEMENT

Project Title: EcoCart – Sustainable Products E-Commerce Order Management System

Skill Task: DBMS

Week: 6

Objectives

1. Create Review and Rating tables.
2. Store customer feedback and ratings.
3. Retrieve product review details.
4. Calculate average product ratings using aggregate functions.
5. Identify highly rated products.

1. Database Design

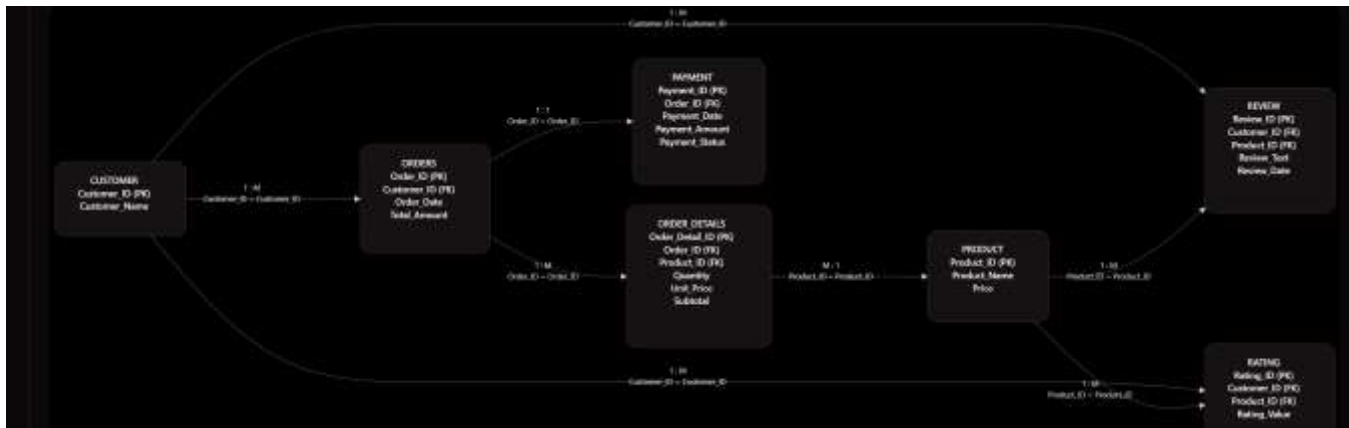
1.1 Review Table

<i>Attribute</i>	<i>Data Type</i>	<i>Constraint</i>
<i>Review_ID</i>	INT	Primary Key
<i>Customer_ID</i>	INT	Foreign Key
<i>Product_ID</i>	INT	Foreign Key
<i>Review_Text</i>	VARCHAR2(500)	NOT NULL
<i>Review_Date</i>	DATE	NOT NULL

1.2 Rating Table

<i>Attribute</i>	<i>Data Type</i>	<i>Constraint</i>
<i>Rating_ID</i>	INT	Primary Key
<i>Customer_ID</i>	INT	Foreign Key
<i>Product_ID</i>	INT	Foreign Key
<i>Rating</i>	INT	NOT NULL
<i>Rating_Date</i>	DATE	NOT NULL

2. Table Relationships



3. Table Creation

3.1 Create Review Table

```
SQL> CREATE TABLE Review ( Review_ID NUMBER(5) PRIMARY KEY, Customer_ID NUMBER(5), Product_ID NUMBER(5), Review_Text VARCHAR2(500) NOT NULL, Review_Date DATE NOT NULL,CONSTRAINT fk_review_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),CONSTRAINT fk_review_product FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID) );
```

Table created.

3.2 Create Rating Table

```
SQL> CREATE TABLE Rating ( Rating_ID NUMBER(5) PRIMARY KEY, Customer_ID NUMBER(5), Product_ID NUMBER(5), Rating NUMBER(1) NOT NULL, Rating_Date DATE NOT NULL,CONSTRAINT fk_rating_customer FOREIGN KEY (Customer_ID) REFERENCES Customer(Customer_ID),CONSTRAINT fk_rating_product FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID) );
```

Table created.

4. Review Data Insertion

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date) VALUES (601, 101, 101, 'Good quality and eco-friendly product', SYSDATE);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date) VALUES (602, 102, 102, 'Useful product with good quality', SYSDATE);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date) VALUES (603, 101, 101, 'Very satisfied with the product', SYSDATE);
```

1 row created.

```
SQL> INSERT INTO Review (Review_ID, Customer_ID, Product_ID, Review_Text, Review_Date) VALUES (604, 103, 103, 'Good product and worth the price', SYSDATE);
```

1 row created.

5. Rating Data Insertion

```
SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date) VALUES (701, 101, 101, 5, SYSDATE);
1 row created.

SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date) VALUES (702, 102, 102, 4, SYSDATE);
1 row created.

SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date) VALUES (703, 103, 103, 5, SYSDATE);
1 row created.

SQL> INSERT INTO Rating (Rating_ID, Customer_ID, Product_ID, Rating, Rating_Date) VALUES (704, 104, 104, 3, 4, SYSDATE);
1 row created.
```

6. Retrieve Product Review Details

```
SQL> SELECT r.Review_ID, c.Customer_Name, r.Product_ID, r.Review_Text, r.Review_Date FROM Review r JOIN Customer c ON r.Customer_ID = c.Customer_ID ORDER BY r.Review_Date DESC;
```

REVIEW_ID
CUSTOMER_NAME
PRODUCT_ID
REVIEW_TEXT
REVIEW_DA
604
Karthik S
103
REVIEW_ID
CUSTOMER_NAME
PRODUCT_ID
REVIEW_TEXT

7. Calculate Average Product Ratings

This uses the AVG() aggregate function.

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Rating GROUP BY Product_ID;
```

PRODUCT_ID	AVERAGE_RATING
102	4
101	5
103	4

8. Identify Highly Rated Products

Products with an average rating of 4 or above can be displayed using HAVING.

```
SQL> SELECT Product_ID, AVG(Rating) AS Average_Rating FROM Rating GROUP BY Product_ID HAVING AVG(Rating) >= 4 ORDER BY Average_Rating DESC;
```

PRODUCT_ID	AVERAGE_RATING
101	5
103	4
102	4

9. Product Rating Report

A combined report can be generated using the Rating and Product tables.

```
SQL> SELECT Product_ID, COUNT(Rating_ID) AS Total_Ratings, AVG(Rating) AS Average_Rating, MAX(Rating) AS Highest_Rating, MIN(Rating) AS Lowest_Rating FROM Rating GROUP BY Product_ID ORDER BY Average_Rating DESC;
```

PRODUCT_ID	TOTAL_RATINGS	AVERAGE_RATING	HIGHEST_RATING	LOWEST_RATING
101	2	5	5	5
103	1	4	4	4
102	1	4	4	4